

A Symbolic–Probabilistic Dialogue System

Peter Mekis

mekis.peter@gmail.com

30 September 2026

Project summary

Current LLMs can generate plausible interpretations and conclusions without an explicit, checkable account of how an utterance changes the task or what follows from it. Across sustained dialogue, they may settle silently on the likeliest reading, lose or contradict earlier requirements, and perform unevenly across domains. These limitations may be tolerable in ordinary conversation but can undermine technical and scientific work, where a misinterpreted requirement or invalid step can invalidate a result. A related risk is model collapse: under some conditions, repeated training on generated text can narrow learned distributions and propagate errors, especially when it replaces independently sourced data.

The project proposes a dialogue system based on compositional dynamic semantics and an explicit information state. LLMs translate between natural language and candidate formal interpretations; a symbolic engine applies state transitions and checks their consequences. The user's active participation is necessary to establish intended meaning: the system should ask when no candidate is adequate or plausible readings imply materially different updates, rather than treating the most likely reading as confirmed. When exhaustive exploration of formal state transitions is impractical, learned search guides which possibilities to examine while symbolic rules determine validity. The project will explore whether the formal layer can provide a partial check against model-collapse risks by verifying represented properties of synthetic training examples before reuse.

The first benchmark would be dialogue-driven development of applications implementing a complex distributed protocol, where evolving requirements and simulated message faults make correctness testable. It would assess whether the architecture supports precise problem solving through sustained dialogue and adds value over LLM-only coding and LLM-assisted coding with conventional testing or verification tools.

The project will use existing LLMs and established search methods, rather than train a new foundation model. The initial prototype will use conventional search; learned, domain-specific guidance could be explored later if results warrant it. The project draws on two distinct foundations: my academic work in dynamic semantics and my pre-generative-AI experience developing a logic-based chatbot.

1. THE PROBLEM AND THE OPPORTUNITY

Large language models can discuss complex problems, help solve them, and generate increasingly complex code, but fluency does not guarantee that they have captured the user's intent or maintained the requirements needed for a correct result. They generate interpretations and conclusions from learned patterns without providing an explicit, checkable account of how each utterance updates the task or what follows from it. Faced with ambiguity, they may proceed with the most plausible reading without exposing alternatives; across sustained dialogue, they may lose, misapply or contradict accumulated requirements. Since intention cannot always be determined from language and context alone, precision also requires the user's active participation. Treating a model's best guess as established intent can turn an unconfirmed interpretation into a hidden design decision.

These limitations may be tolerable in much ordinary conversation, but in technical and scientific work a misinterpreted requirement or invalid step can undermine a specification, calculation or conclusion. Model performance also varies across domains; fluency does not guarantee reliable reasoning in a specialized field. A related, longer-term concern is model collapse: under some training conditions, recursive use of generated text can narrow learned distributions and propagate errors [1].

Symbolic systems offer an important counterpoint: they can represent constraints explicitly and check whether a step follows from them. But building them often requires developers to encode language and domain knowledge by hand. This creates a substantial knowledge-engineering burden and makes it costly to adapt systems to new tasks. Even when the relevant states and rules have been specified, exhaustive reasoning can become computationally impractical as the number of possible sequences of steps grows.

The project investigates a division of labour: probabilistic models interpret language and guide search, while formal semantics and symbolic machinery represent dialogue state and check consequential reasoning. The aim is not to formalize every utterance or replace existing language models, but to test whether an explicit, revisable information state can support more precise problem solving when tasks depend on accumulated requirements and verifiable outcomes.

Key takeaway

The project addresses a gap between fluent dialogue and reliable multi-turn reasoning. It combines probabilistic language processing with explicit dialogue state, symbolic checks, and guided search, then tests that combination in a domain with verifiable outcomes.

2. THE PROPOSED ARCHITECTURE

The architecture of the proposed system combines three components: a large language model for interpretation and expression, a symbolic engine for formal state updates and verification, and a search engine for computationally difficult reasoning, with learned guidance as a later development. The formal approach draws on my academic work in dynamic semantics, while the symbolic engine also draws on a logic-based chatbot I devel-

oped before the widespread use of generative AI. Integrating these foundations with existing language models and search methods is new work.

The project would use existing language models; it does not require developing or training a new foundation model. Development would concentrate on the formal representation, symbolic operations and integration of the components. The first prototype will use conventional search. Training a smaller, domain-specific model to guide that search would be a later step, subject to experimental results and available resources.

The formal language and information state. The formal language is a defined fragment of ordinary English with compositional dynamic semantics. Its expressions describe operations on an information state: an assertion adds information, a question introduces an issue to resolve, and a correction revises an earlier commitment. The effect of a complex expression is determined by its structure and the meanings of its parts. This gives the system a formal account of both the subject being discussed and the dialogue through which it is established.

The information state records requirements, assumptions, objects and relations, together with current questions, unresolved references and commitments. It also records the source and status of information. Something asserted by the user, retrieved from an external source or derived from premises must remain distinguishable. Formal reasoning establishes consequences of premises; it does not establish the truth of those premises merely by using them. A possible extension is to use symbolic checks on represented properties of synthetic training examples before reuse.

Designing the representation of these states and developing suitable database models for storing and updating them are open tasks. The representation must support the compositional dynamic semantics, including dependencies needed to revise earlier conclusions. No particular database technology or representational scheme has yet been selected.

From ordinary language to formal interpretation. The language model proposes candidate readings in the formal fragment. This involves identifying matching patterns, resolving references such as “that field” or “the previous assumption”, and ranking the resulting interpretations. A reading includes its structure and reference assignments, so identical wording can still correspond to different formal interpretations.

Disambiguation takes place over these formal candidates. The symbolic engine checks their compatibility with the information state and compares their consequences. Readings that can be shown to be equivalent need not be treated as separate choices. Where plausible readings produce materially different updates and context does not settle the intended meaning, the system asks the user. It should also ask when no candidate provides a sufficient basis for proceeding.

Symbolic processing and the search loop. Once an interpretation has been sufficiently established, the symbolic engine applies the corresponding operation. A straightforward update may be performed directly. A difficult inference, plan or construction may require exploring many possible sequences of operations.

In that case, the symbolic engine and search engine work in

a loop. The search engine proposes a step to explore. The symbolic engine checks whether the step is permitted and computes its effect. The search engine then evaluates the resulting state and decides whether to continue along that path or explore another. Candidate paths remain provisional until a result has passed the required checks.

If learned guidance is added, a neural component would estimate promising steps and intermediate states. The symbolic engine would define what those steps mean and whether they are valid. Learned estimates would therefore influence which paths are explored without determining logical validity. This is the intended role of an AlphaZero-inspired mechanism; it would not guarantee that a solution will be found. Its operations and evaluation criteria must be chosen for the application domain.

From information state to output. For a conversational response, the symbolic engine determines the formal content and appropriate dialogue move. A prompt generator selects the relevant facts, commitments and constraints from the information state for an existing language model to express in natural language. For a requested task-specific output, the system can use a symbolic generator where one is available. Any guarantees from such a generator are limited to properties represented in its formal model and depend on the model and generator being sound. Otherwise, the information state can guide a request to an existing language model or other probabilistic engine. This makes established requirements explicit and may reduce omissions or drift compared with generation from the dialogue alone, but does not guarantee that the resulting response or artifact conforms to them; that fidelity must be assessed.

The overall process is therefore language-model interpretation, symbolic processing—with a search loop when needed—and output generation suited to the task. A clarification question returns the process to the user, whose answer is interpreted against the same persistent information state.

Key takeaway

Use existing language models for interpretation and expression. Give utterances an explicit meaning as changes to an information state. Use the search engine to choose what to explore and the symbolic engine to compute and check each step. Ask when the intended update remains unclear. Preserve the formal result when expressing it in ordinary language.

3. A FIRST BENCHMARK: DIALOGUE-DRIVEN DEVELOPMENT OF DISTRIBUTED SOFTWARE

Software that depends on complex distributed protocols offers a demanding benchmark for dialogue-driven development. A first prototype of the proposed system would act as a dialogue-based software development assistant: the user would describe and revise an application's requirements, while the chatbot maintains a formal specification and generates and updates the code.

The initial benchmark would be an application implementing a complex distributed protocol, so its behavior could be tested under simulated faults. The generated application could

be cross-platform, with Flutter as one possible target framework. The evaluation would test whether the chatbot preserves protocol requirements as the user develops and revises the application through sustained dialogue.

Distributed protocols must coordinate state across processes despite failures and unpredictable communication. Messages may be delayed, lost, duplicated or reordered; processes may crash and later reconnect. Small changes in the requirements can alter what counts as correct. “The latest edit wins,” for example, could mean the edit with the latest timestamp, the latest one received by a server, or the edit that causally follows another.

The system would represent the protocol requirements and relevant application state explicitly. It would translate each request into candidate formal updates, check their effects on the current specification, and ask for clarification when plausible readings imply materially different behavior. Once the intended update is established, it would explore possible execution sequences and test or model-check candidate implementations against the required properties under the specified fault model. Claims would be limited to the explored executions unless the relevant state space is exhaustively verified.

For protocol-critical tasks with sufficiently precise formal specifications, the project would develop a task-specific deterministic code generator for critical components. It would generate this code directly from the formal specification. Its guarantees would depend on the soundness of the generator and the adequacy of the specification. For other application code, an existing language model would generate or revise code using prompts built from the current information state and the relevant codebase. This makes established constraints explicit at each step and may reduce drift, but does not ensure that the generated code conforms to them.

The symbolic layer would derive code-review obligations and required tests from the formal constraints, then prompt the language model to review affected code and implement the required tests. It would determine which properties must be checked, rather than leaving the language model to invent the test obligations. Generated tests would be run against the application in the local simulator, and their results checked against the requirements. This adds a safety layer, though it cannot establish correctness beyond the properties and cases that the formal model and tests cover.

Code quality is part of the benchmark alongside correctness. As the application evolves, the generated code should remain readable and maintainable, with concerns kept separate, unnecessary duplication avoided, and new changes integrated into the existing architecture.

The first benchmark should restrict the protocols and fault assumptions. It could include tasks based on leader election, atomic commit, or a narrowly scoped consensus protocol, with specified safety properties such as agreement or all-or-nothing outcomes. A local simulator could inject message loss, delay, duplication, reordering and process crashes, making the benchmark reproducible on a single machine. The initial prototype would use conventional search to explore execution paths. Learned guidance could later prioritize paths if results warrant it; conclusions would be limited to the explored state space

unless the relevant space is exhaustively checked or a more general result is established.

Evaluation would test the whole interaction, not just whether the generated code compiles. Tasks could introduce new requirements, corrections and ambiguous changes over several turns. Measures could include whether the system preserves established constraints, asks when ambiguity affects correctness, finds protocol violations, passes specified checks, and maintains readable, maintainable code as the application changes. Comparisons with LLM-only coding and LLM-assisted coding using conventional testing or verification tools would show whether the proposed architecture adds value. If learned guidance is later introduced, it would also be compared with conventional search over the same symbolic model.

This would test a sharper claim than general app generation: whether explicit dialogue state, symbolic verification, search-based exploration and code generation tied to formal requirements can help an AI system develop software whose correctness depends on complex distributed behavior while keeping the code understandable and maintainable.

Key takeaway

Use complex distributed protocols as a demanding benchmark for dialogue-driven development. The system should preserve evolving requirements, clarify consequential ambiguities, generate protocol-critical code from formal specifications, and use those specifications to guide other code generation, review and fault testing while maintaining readable, maintainable code.

4. EXISTING RESEARCH

Although the proposed architecture takes a different approach from dialogue systems centered on LLM generation, it builds on established research. Prior work has explored explicit dialogue-state updates, ambiguity resolution, learned search through formal states, and the testing of LLM-generated distributed protocols. Together, these studies ground the proposal and frame its broader question: whether these strands can be integrated into a dialogue system for precise, sustained problem solving.

Maraev, Berman and Larsson’s SISU system combines Information State Update rules, Harel statecharts and LLMs. It demonstrates how explicit dialogue state and rule-based control can support flexible but controllable form-filling conversations. This is a close precedent for representing discourse as structured state that changes through dialogue. Its focus is dialogue management, rather than learned search over formal states or distributed protocol development [2].

Saparina and Lapata’s “Disambiguate First, Parse Later” generates alternative interpretations before translating a request into SQL. Their work addresses the tendency of language models to settle prematurely on one reading. The proposed system shares the aim of preserving plausible alternatives, while placing disambiguation at the formal interpretation layer: candidate readings are compared by their consequences for the information state, and the user is asked when those consequences differ materially [3].

AlphaProof demonstrates learned search through formal

mathematical states. A neural model guides the search through Lean proof states, while Lean checks the resulting proofs. This provides a strong precedent for using learned guidance to navigate a symbolic search space without making validity depend on the learned model. It leaves open how such methods might extend to dialogue states and other domains, including software development [5].

Das and Coyne’s 2026 paper, “Testing LLM-Generated Distributed Protocol Code,” evaluates generated implementations of two-phase commit, ring election and Raft under simulated message loss, delay and duplication. The authors report that LLMs struggle with complex consensus algorithms and debug them inconsistently. This provides a relevant test setting for the proposed architecture, though their work evaluates protocol implementations rather than dialogue-driven reasoning [4].

These studies establish precedents for important components, but leave open whether they can be integrated into a dialogue system that maintains and updates a formal information state, clarifies consequential ambiguity, and uses learned search with symbolic verification.

The broader research question is whether this combination can support precise, sustained problem solving across multiple turns. Distributed protocol development is one proposed test application: it would let us evaluate the dialogue system against demanding, verifiable tasks without defining the project’s overall scope.

Key takeaway

Prior work grounds the components individually: structured dialogue state, ambiguity-aware semantic parsing, learned search with formal verification, and fault-based evaluation of LLM-generated distributed protocols. The research question is whether these can be integrated into a dialogue system for precise, sustained problem solving.

5. LIMITS AND OPEN QUESTIONS

The project would use existing language models; it does not propose training a new foundation model. The research lies in combining probabilistic language processing with formal dialogue-state representation, symbolic reasoning and guided search. This keeps the model-development scope bounded, though the cost of building the symbolic components, gathering useful search experience and evaluating the integrated system remains open.

The project does not aim to formalize unrestricted natural language. A prototype will need a bounded semantic fragment and a restricted task domain. The central design challenge is to make the formal language expressive enough to represent meaningful requirements while keeping interpretation and reasoning tractable. The information-state representation and database model also remain open. They must capture both task facts and discourse, including provenance and dependencies needed when earlier assumptions or conclusions are revised. For the distributed-software benchmark, the system must also relate formal requirements to the evolving codebase so that changes can target affected components without losing established constraints.

Formal verification cannot compensate for an incorrect interpretation. The language model may omit the intended reading or rank it incorrectly, and the symbolic engine can still reason correctly from the wrong formalization. Logic alone cannot determine what the user meant. The system must therefore be evaluated on candidate coverage, consequential ambiguity and its decisions to ask for clarification. Facts about the world also require external grounding; a conclusion derived from an uncertain premise remains conditional on that premise. Likewise, a sound deterministic generator can guarantee only properties represented in its formal model, while code and tests produced with probabilistic models remain fallible and require independent checks.

Learned search can guide exploration, but it does not remove combinatorial complexity or guarantee that a solution will be found. The useful states, actions and evaluation criteria will depend on the task. Designing a search space that is rich enough to test the architecture, yet small enough to implement and evaluate, is itself a major research question. The first prototype will use conventional search; whether learned guidance adds value, and how it could be trained affordably, should be established experimentally.

Verification is always relative to a model, specification and fault assumptions. Passing tests against simulated message loss or delay would establish properties within that test setup; it would not prove that the formalized requirements capture the user's intent or cover every real-world behavior. The distributed-protocol benchmark can test the architecture in a demanding domain, but results there would not by themselves establish that it generalizes to dialogue systems as a whole. The proposed check on synthetic training examples would be similarly limited: it could verify only properties represented in the formal language, not unrepresented errors or distributional diversity. It should be treated as an exploratory partial check, not protection against model collapse.

The proposed contribution also requires careful comparison with prior work: the research question concerns the integration and measured behavior of established ideas, not the invention of each component separately.

Key takeaway

The system will use existing LLMs and a bounded formal fragment. Its reliability remains limited by what the parser represents, what the symbolic model covers, and what search and evaluation can establish.

6. PROTOTYPE AND EVALUATION

The first incubation phase would test whether the proposed architecture can support precise, multi-turn development of software implementing a distributed protocol. The prototype would use existing language models and run its protocol simulator and verification tools on a single machine.

Work would proceed in stages. First, the team would define a bounded benchmark around one protocol and an explicit fault model. Then it would build the dialogue layer needed to represent requirements, track revisions, generate formal interpretations and ask for clarification when different readings

imply different behavior. A symbolic model and simulator would check protocol transitions under specified faults. The prototype would develop a deterministic generator for protocol-critical code and use an existing language model to generate or revise other code from the information state and codebase. The symbolic layer would derive review obligations and required tests from the formal constraints, then prompt the language model to review affected code and implement those tests; the simulator would run the tests under the specified faults. The initial search will be conventional; learned guidance would be added only if the prototype establishes a useful search task and produces experience from which it could learn.

Evaluation would compare the system with LLM-only development and LLM-assisted development using conventional testing or verification tools. Measures would include preservation of requirements through dialogue, protocol violations under simulated faults, useful versus unnecessary clarification, code quality as the application changes, and the time and computational effort needed to reach a checked result. The project would proceed to broader protocols only if the initial benchmark showed that the architecture handles evolving requirements and failure scenarios well enough to justify the added complexity.

Key takeaway

Begin with one protocol under explicit fault assumptions, then evaluate multi-turn requirement updates and verified behavior against established baselines. Expand only if initial results show that the hybrid architecture adds value.

7. DEVELOPMENT FOUNDATION AND PATH TO A VENTURE

The project draws on two distinct foundations. My academic work in dynamic semantics informs the formal language and dialogue-state design. Separately, my pre-generative-AI work developing a logic-based chatbot, including a Haskell prototype, provides a starting point for the symbolic components. An initial technical review would identify which parts can be reused and which need redesign. The project would then integrate these foundations with existing language models and a search-and-verification loop. My background combines research in logic with practical software development.

The initial product hypothesis is an engineering environment that lets software teams express protocol requirements in dialogue, maintain a formal specification as those requirements change, generate protocol-critical code from that specification, and use it to guide generation, review and testing of the wider application. Distributed-protocol development is a narrow first use. If the approach proves useful, the product could grow toward a broader tool for specifying, generating, testing and maintaining complex software systems.

That commercial path remains to be tested. Incubation should establish whether engineers face this problem often enough to adopt a dedicated tool, where it would fit into their existing development process, and whether its verification capabilities provide value beyond current coding assistants and testing tools. The project seeks support for a staged proof of con-

cept that can answer those technical and commercial questions and determine whether a viable startup can emerge.

Key takeaway

Build from prior formal chatbot work, assess what can be reused, and test whether a protocol-focused engineering assistant solves a problem developers will adopt. A startup path depends on both technical results and customer validation.

■ REFERENCES

- [1] I. Shumailov et al., “Ai models collapse when trained on recursively generated data”, *Nature*, vol. 631, pp. 755–759, 2024. DOI: 10.1038/s41586-024-07566-y [Online]. Available: <https://www.nature.com/articles/s41586-024-07566-y>
- [2] V. Maraev, A. Berman, and S. Larsson, “Combining information state update, harel statecharts and LLMs for controllable and flexible conversational AI”, in *Proceedings of the 2025 CLASP Conference on Language Models and Representations (LARP)*, Gothenburg, Sweden: Association for Computational Linguistics, 2025, pp. 28–37. [Online]. Available: <https://aclanthology.org/2025.clasp-main.3/>
- [3] I. Saparina and M. Lapata, “Disambiguate first, parse later: Generating interpretations for ambiguity resolution in semantic parsing”, in *Findings of the Association for Computational Linguistics: ACL 2025*, Association for Computational Linguistics, 2025, pp. 16 825–16 839. DOI: 10.18653/v1/2025.findings-acl.863 [Online]. Available: <https://aclanthology.org/2025.findings-acl.863/>
- [4] A. Das and B. Coyne, *Testing LLM-generated distributed protocol code*, PAgE 2026, in the PLDI 2026 program, Conference paper abstract, 2026. [Online]. Available: <https://pldi26.sigplan.org/details/page-2026-papers/5/Testing-LLM-Generated-Distributed-Protocol-Code>
- [5] T. Hubert et al., “Olympiad-level formal mathematical reasoning with reinforcement learning”, *Nature*, vol. 651, no. 8106, pp. 607–613, 2026, Published online in 2025. DOI: 10.1038/s41586-025-09833-y [Online]. Available: <https://www.nature.com/articles/s41586-025-09833-y>